

Motorized Travelling Salesman Problem

Shea Schmidt, Katie Deppe, Sam Salita

The motorized travelling salesman problem (MTSP) involves the optimization of trajectory to visit N points in as short of time as possible with limited

control to simulate a vehicle.

This implementation of the MTSP involves choice of origin to one of four corners of a two dimensional grid and 10 target points, hereafter called cities.

Throughout this report, both comments and text are used to discuss results and implementation of functions.

The equations of motion that govern our vehicle are:

EOMs:

$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

$$\dot{v} = a$$

$$\dot{\theta} = \gamma$$

With control limited as:

$$-1 \leq a \leq 1$$

$$-1 \leq \gamma \leq 1$$

This gives 4 state variables and 2 control variables (acceleration and turn rate).

The cost to optimize is essentially then:

$$J = \frac{1}{r} \int_0^{t_f} \alpha^2 + \gamma^2 dt$$

```
tic
clc; clear; close all
```

The agreed definition of 10 cities, origins, and sequences:

```
cities = [6.6 8.5 6.8 7.4 6.6 7.1 2.8 1.0 6.9 9.5;
          0.4 9.3 7.6 3.9 1.7 0.3 0.5 8.2 3.2 0.3];
origin = [0,10,10,0;0,0,10,10];
A = perms(1:10);
sequence =
[ones(length(A),1),A;2*ones(length(A),1),A;3*ones(length(A),1),A;4*ones(length(A),1),A];
```

A genetic algorithm is used to generate a Pareto Optimal Front to determine possible good initial guesses:

```
[x,fval,exitflag,output,population,scores] =
gamultiobj(@FITNESSfunc(x,sequence,cities,origin),1,[],[],[],[],1,14515200,[],1);
```

gamultiobj stopped because the average change in the spread of Pareto solutions is less than options.FunctionTolerance

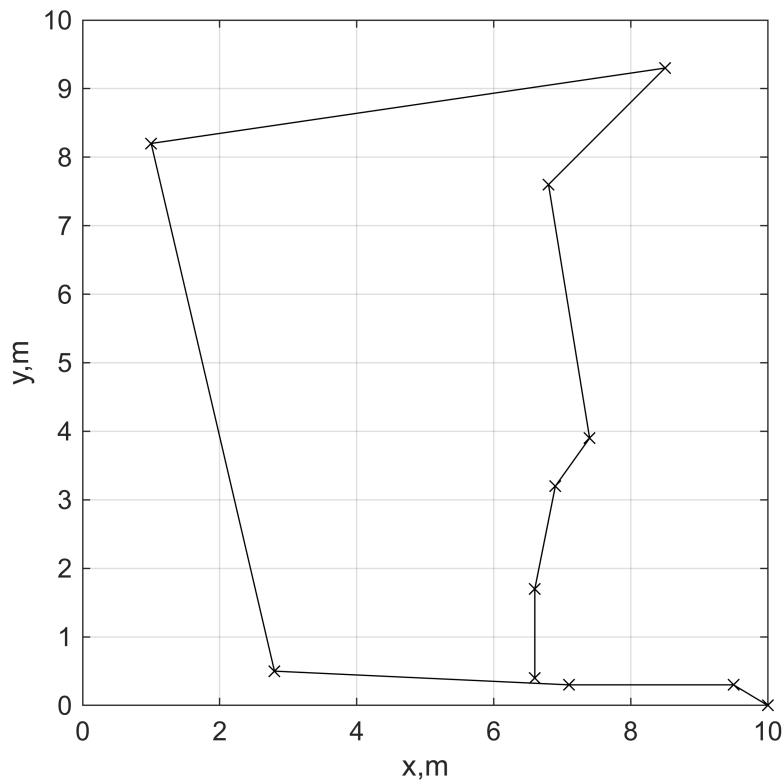
Breaking up the sequence information into the following variables allows for better readability and usage in subfunctions.

```
close all
Goodx = 3761125; % chosen sequence from the GA
order = sequence(Goodx,:);
originx = origin(1,sequence(Goodx,1)); % define origin of sequence
originy = origin(2,sequence(Goodx,1));
citiesx = [originx,cities(1,sequence(Goodx,2:11))]; % define order of cities in
sequence with origin
citiesy = [originy,cities(2,sequence(Goodx,2:11))];
citySeq = [citiesx; citiesy] % create sequence matrix
```

```
citySeq = 2×11
    10.0000    9.5000    7.1000    2.8000    1.0000    8.5000    6.8000    7.4000 ...
         0     0.3000    0.3000    0.5000    8.2000    9.3000    7.6000    3.9000
```

Initial plot of selected sequence draws straight line distance between each city.

```
plot(citiesx, citiesy, '-kx') %plot sequence path
xlabel('x,m') % label for x axis
ylabel('y,m') % y Label for y axis
grid on % grid on for readability
axis equal %set axes equal
axis([0 10 0 10]) % set axis limits
hold on % plot on this figure more
```



Segment I: Origin to City 1 and 2

The first segment begins at the origin and travels to the first two cities in the selected sequence.

A genetic algorithm is used to make a good guess for the unknown variables during critical times on this segment, notably:

$$\theta(0) \quad \lambda_x(0) \quad \lambda_y(0) \quad \lambda_v(0) \quad \lambda_x(t_1^+) \quad \lambda_y(t_1^+) \quad t_1 \quad t_2$$

Which, respectively, refer to initial angle, lambdas for x, y, and v, and both of the switching times. The switching times are times of intercept of city 1 and city 2.

Lambda x and Lambda y are not continuous as the values for x and y are known at each city.

r in the following began set at 2, but ended set to 100 to approximate the true minimum time optimization problem.

```
r = 100;
% options = optimoptions('ga','UseParallel',true,'MaxGenerations',1000);
% x1 = ga(@(x)FITNESS1(x,citySeq,r),8,[],[],[],[], [pi/2,-1*ones(1,5),0.1,0.1],
[pi,1*ones(1,5),3,3],[],[],options)
```

Genetic algorithm guess for states

```
% x1 = [2.6401    -0.3921    -0.7038    -0.8914     0.5414     0.1052     1.1523
1.5274]; % r = 2
```

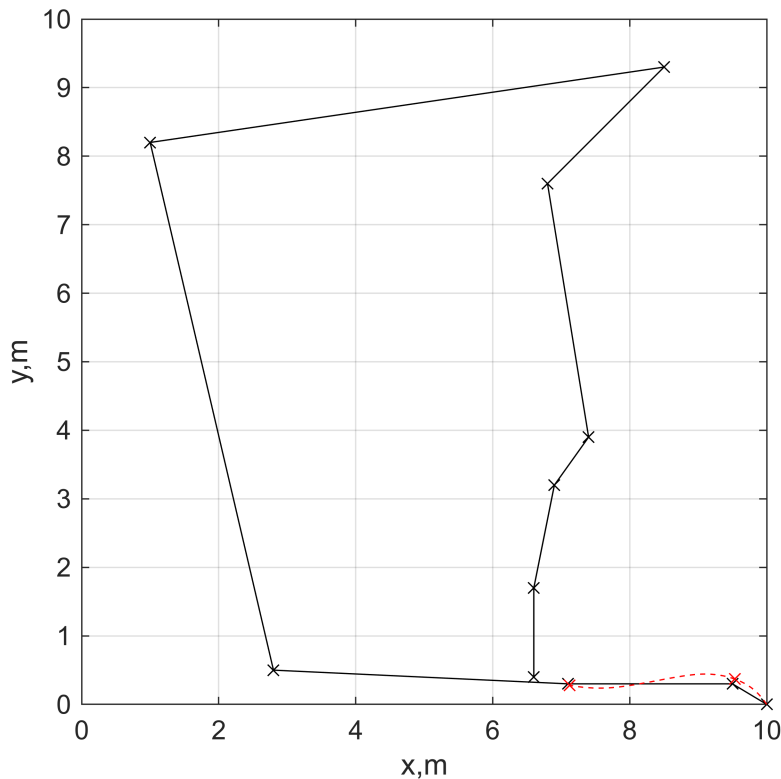
```
x1 = [2.1982    0.1943   -0.3602   -0.9841    0.4138    0.0924    1.1003
1.3680]; % r = 100
```

Guess generation of states, lambdas

```
[T1,S1] = ode45(@(t,s)ELEOM(t,s,r),[0,x1(7)],[originx,originy,0,x1(1:4),0]);
%generate guess origin -> city one
[T2,S2] = ode45(@(t,s)ELEOM(t,s,r),[0,x1(8)],[S1(end,1:4),x1(5:6),S1(end,7:8)]);
%generate guess city one -> city two
```

Plot of guess to first two cities

```
plot([S1(:,1);S2(:,1)],[S1(:,2);S2(:,2)], '--r',...
      [S1(end,1);S2(end,1)],[S1(end,2);S2(end,2)], 'xr') % plot guess for first two
cities
hold off % no longer plot on this figure
```



BVP4C solution generation: Use guess for BVP4C

```
solinit.x = linspace(0,1,11);

arc1 = interp1(T1,S1,solinit.x*T1(end)); % interpolate arc 1
arc2 = interp1(T2,S2,solinit.x*T2(end)); % interpolate arc 2

solinit.y = [arc1,arc2]'; % create states matrix

solinit.parameters = [T1(end),T2(end)]; % create parameters matrix
```

```

sol =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc(so,sf,times,cities,r,
Goodx,sequence,originx,originy),solinit); % bvp4c of first two cities
arc12solx = sol.x; % x of first two cities
arc12soly = sol.y; % states of first two cities
arc12solp = sol.parameters; % times of first two cities

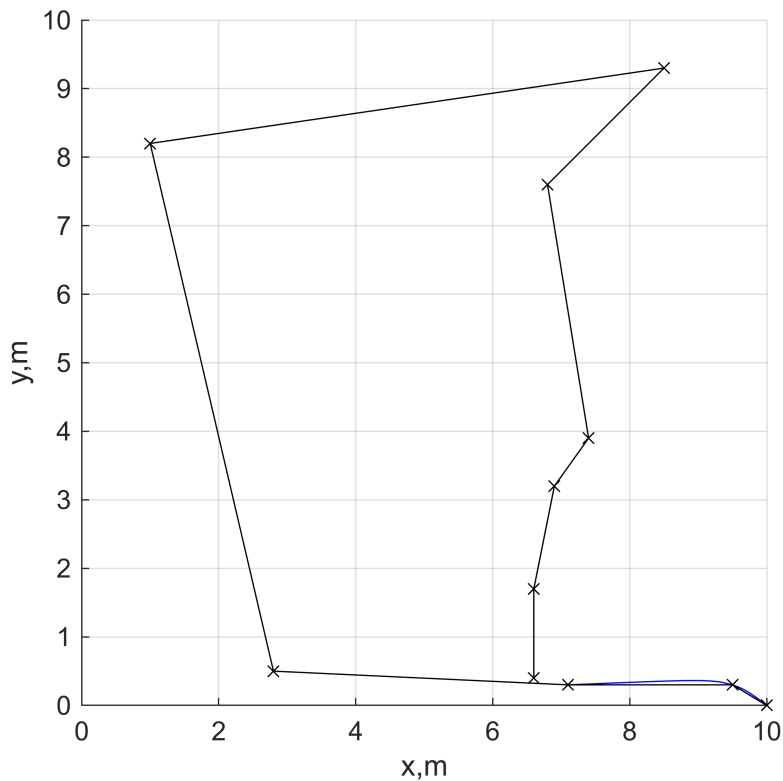
```

Plot of BVP4C solution to first two cities:

```

figure();
hold on
x = [sol.y(1,:),sol.y(9,:)]; % x to first two cities
y = [sol.y(2,:),sol.y(10,:)]; % y to first two cities
plot(x,y,'b-') % plot solution to first 2 cities
plot([origin(1,sequence(Goodx,1)),cities(1,sequence(Goodx,2:11))],...
      [origin(2,sequence(Goodx,1)),cities(2,sequence(Goodx,2:11))],'-kx') % plot
cities
xlabel('x,m') % label x axis
ylabel('y,m') % label y axis
grid on % grid on for readability
axis equal % set axes equal
axis([0 10 0 10]) % set axis limits
hold off % no longer plot on this figure

```



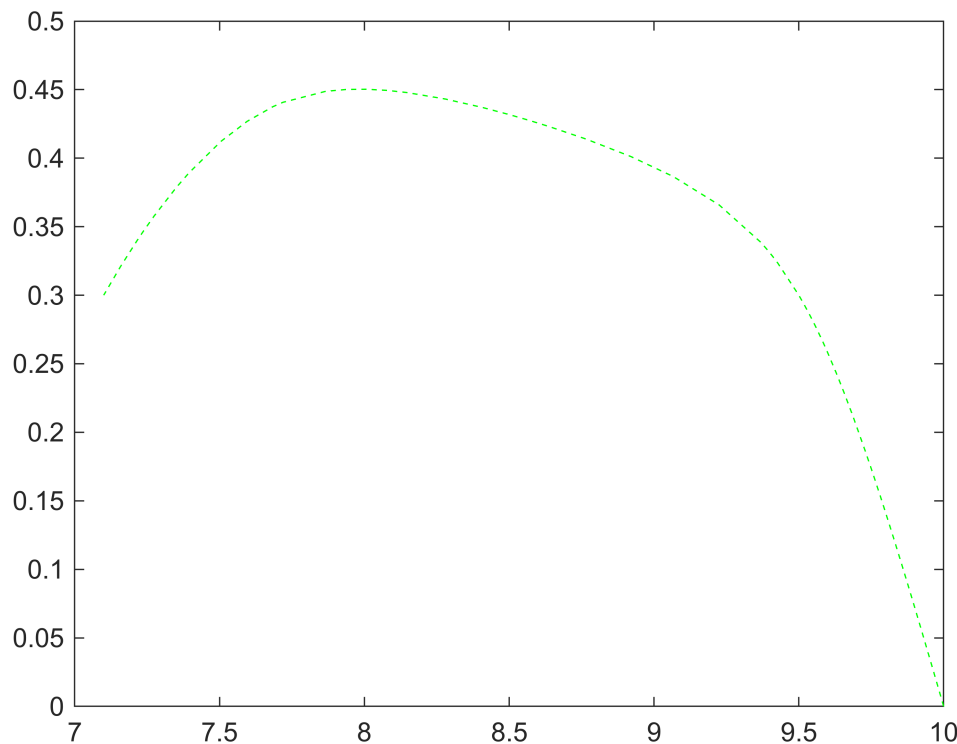
Homotopy is used to reduce curvature to allow for more curated exit angle.

This method iterates bvp4c call using the last solution as the guess for the next iteration.

```
sol1 = sol;
for thetaf = 3.18:0.01:3.5
    sol1 =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc2(so,sf,times,cities,r
,Goodx,sequence,thetaf,originx,originy),sol1);
end
sol = sol1;
x = [sol.y(1,:),sol.y(9,:)]; % homotopy x
y = [sol.y(2,:),sol.y(10,:)]; % homotopy y
```

Plot of homotopy solution:

```
plot(x,y,'g--') % plot of homotopy
```



Segment II: City 2 to City 3 and 4

GA guess of switching times and lambdas is used similarly here, except with two less variable (theta and lambda v) as these are known at

the beginning of this segment.

$\lambda_x(0)$ $\lambda_y(0)$ $\lambda_x(t_1^+)$ $\lambda_y(t_1^+)$ t_1 t_2

```
% options = optimoptions('ga','UseParallel',true,'MaxGenerations',1000);
```

```
% x2 = ga(@(x)FITNESS2(x,citySeq,r,[sol.y(11:16,end)]'),6,[],[],[],[],
[-10*ones(1,4),0.1,0.1],[10*ones(1,4),10,10],[],[],options) %beq and lb
% x2 = [-0.2756    0.1038    -0.0060    0.1459    2.2131    2.7548]; % r = 2
x2 = [-0.0965    0.1331    -0.0517    -0.1878    1.5563    1.9669]; % r = 100
```

Generation of states guess to third and fourth city

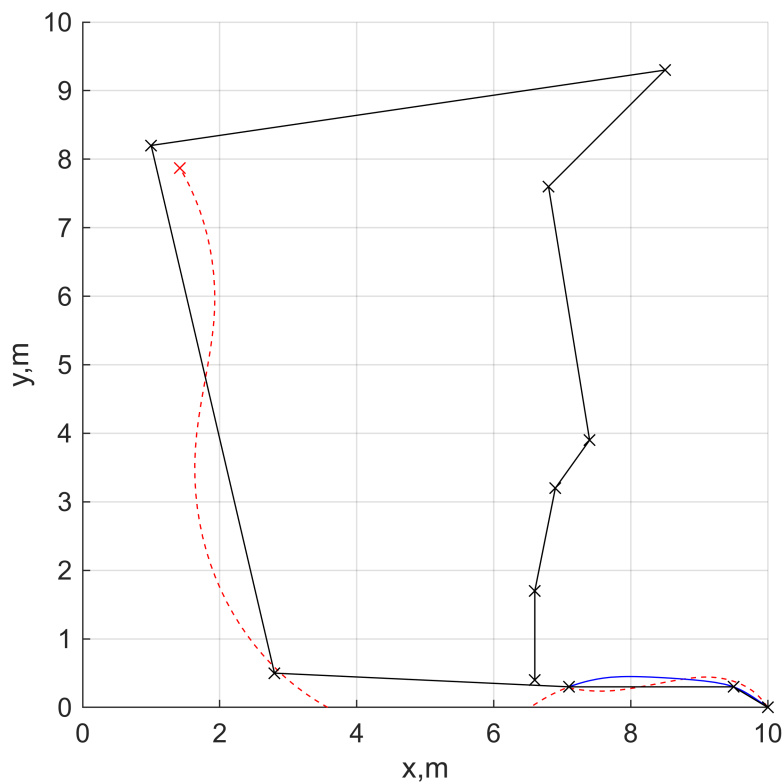
```
[T3,S3] = ode45(@(t,s)ELEOM(t,s,r),[0,x2(5)],[[sol.y(9:12,end)]',x2(1:2),
[sol.y(15:16,end)]']); % guess from 2-3
[T4,S4] = ode45(@(t,s)ELEOM(t,s,r),[0,x2(6)],[S3(end,1:4),x2(3:4),S3(end,7:8)]); %
guess from 3-4
```

Appending states to path to first and second city

```
SALL = [S1;S2;S3;S4];
```

Plot of guess to 3rd and 4th cities

```
figure();
hold on
plot(SALL(:,1),SALL(:,2),'--r') % plot them all together
plot(SALL(end,1),SALL(end,2),'xr') % plot final position
x = [sol.y(1,:),sol.y(9,:)]; % x array to 4ths city guess
y = [sol.y(2,:),sol.y(10,:)]; % y array to 4th city guess
plot(x,y,'b-') % plot guess 2-3-4 cities
plot([origin(1,sequence(Goodx,1)),cities(1,sequence(Goodx,2:11))],...
[origin(2,sequence(Goodx,1)),cities(2,sequence(Goodx,2:11))],'-kx') % plot
cities
xlabel('x,m')% label x axis
ylabel('y,m') % label y axis
grid on % grid on for readability
axis equal % set axes equal
axis([0 10 0 10]) % set axis limits
hold off % no longer plot on this figure
```



BVP4C solution generation for 3rd and 4th citise

```
solinit.x = linspace(0,1,11);
```

Interpolation of next two arcs to 3rd and 4th cities

```
arc3 = interp1(T3,S3,solinit.x*T3(end)); % interpolation of arc 3
arc4 = interp1(T4,S4,solinit.x*T4(end)); % interpolation of arc 4
arc12soly = interp1(arc12solx', arc12soly', solinit.x*arc12solx(end)');
```

Appending 3rd and 4th states guess to previous solutions

```
solinit.y = [arc12soly,arc3,arc4]';
```

Appending 3rd and 4th times to previous solutions

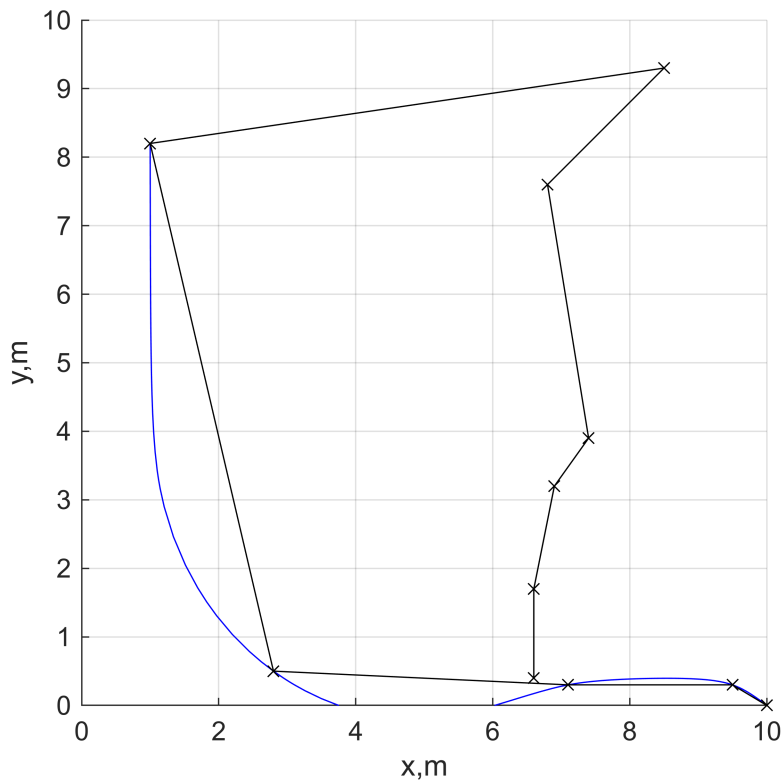
```
solinit.parameters = [arc12solp(1,1),arc12solp(2,1),T3(end),T4(end)];
```

BVP4C Solution to 4th city

```
sol =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc(so,sf,times,cities,r,
Goodx,sequence,originx,originy),solinit);
arc34solx = sol.x; % x 2-3-4
arc34soly = sol.y; % states to 2-3-4
arc34solp = sol.parameters; % times to 4th city
```


Plot of solution to 4th city

```
figure();
hold on
x = [sol.y(1,:),sol.y(9,:),sol.y(17,:),sol.y(25,:)]; % x array to 4th city
y = [sol.y(2,:),sol.y(10,:),sol.y(18,:),sol.y(26,:)]; % y array to 4th city
plot(x,y,'b-') % plot solution to 4th city
plot([origin(1,sequence(Goodx,1)),cities(1,sequence(Goodx,2:11))],...
      [origin(2,sequence(Goodx,1)),cities(2,sequence(Goodx,2:11))],'-kx') % plot
cities
xlabel('x,m') % label x axis
ylabel('y,m') % label y axis
grid on % grid on for readability
axis equal % set axes equal
axis([0 10 0 10]) % set axis limits
```



Segment III: City 4 to City 5 and 6

GA guess of switching times and lambdas

$\lambda_x(0)$ $\lambda_y(0)$ $\lambda_x(t_1^+)$ $\lambda_y(t_1^+)$ t_1 t_2

```
% options = optimoptions('ga','UseParallel',true,'MaxGenerations',1000);
% x3 = ga(@(x)FITNESS3(x,citySeq,r,[sol.y(end-5:end,end)]'),6,[],[],[],[],
[-10*ones(1,4),0.1,0.1],[10*ones(1,4),10,10],[],[],options) %beq and lb
```

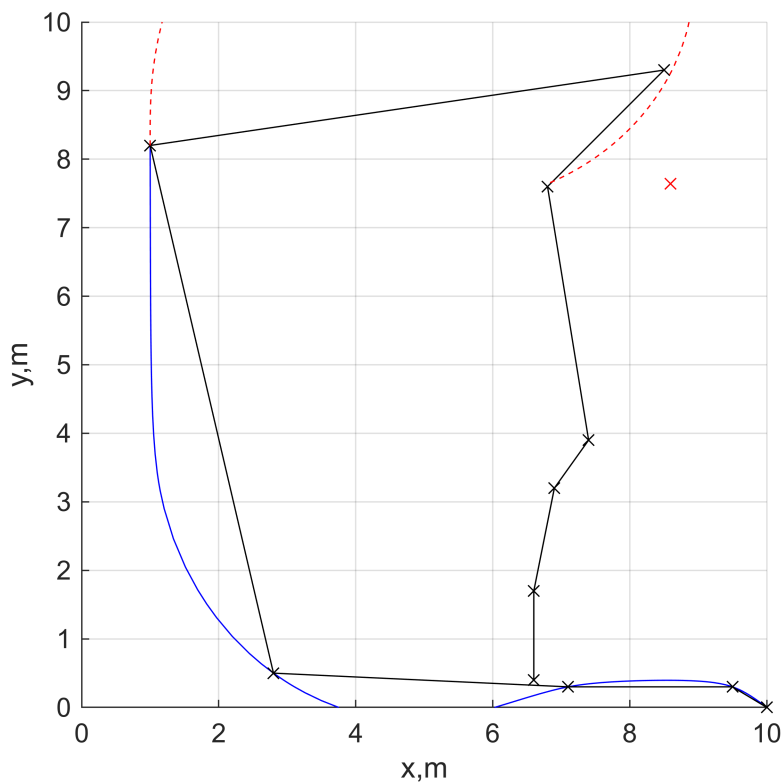
```
% x3 = [0.1404 -0.1964 0.6053 -0.3222 4.2342 0.6409];
x3 = [0.0212    -0.1752    0.4247   -0.3681    3.7023    0.6955]; % r = 100
```

Generation of states guess

```
[T5,S5] = ode45(@(t,s)ELEOM(t,s,r),[0,x3(5)],[[sol.y(end-7:end-4,end)]',x3(1:2),
[sol.y(end-1:end,end)]')]; % generate states guess for 4-5
[T6,S6] = ode45(@(t,s)ELEOM(t,s,r),[0,x3(6)],[S5(end,1:4),x3(3:4),S5(end,7:8)]); %
generate states guess for 5-6
```

Plot of guess

```
plot([S5(:,1);S6(:,1)],[S5(:,2);S6(:,2)], '--r') % plot of guess
plot(S5(end,1),S6(end,2), 'xr')
hold off % no longer plot on this figure
```



BVP4C solution generation

```
solinit.x = linspace(0,1,11);
```

Interpolation of next two arcs

```
arc5 = interp1(T5,S5,solinit.x*T5(end)); %interpolation of arc 5
arc6 = interp1(T6,S6,solinit.x*T6(end)); %interpolation of arc 6
arc34soly = interp1(arc34solx', arc34soly', solinit.x*arc34solx(end)');
```

Appending states guess to previous solution

```
solinit.y = [arc34soly,arc5,arc6]';
```

Appending times

```
solinit.parameters = [arc34solp(1:4,1)',T5(end),T6(end)];
```

Solution

```
sol =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc(so,sf,times,cities,r,
Goodx,sequence,originx,originy),solinit);
arc56solx = sol.x; % x of 4-5-6 sln
arc56soly = sol.y; % states of 4-5-6 sln
arc56solp = sol.parameters; % times of 4-5-6 sln
```

Plot of BVP4C solution 4-5-6

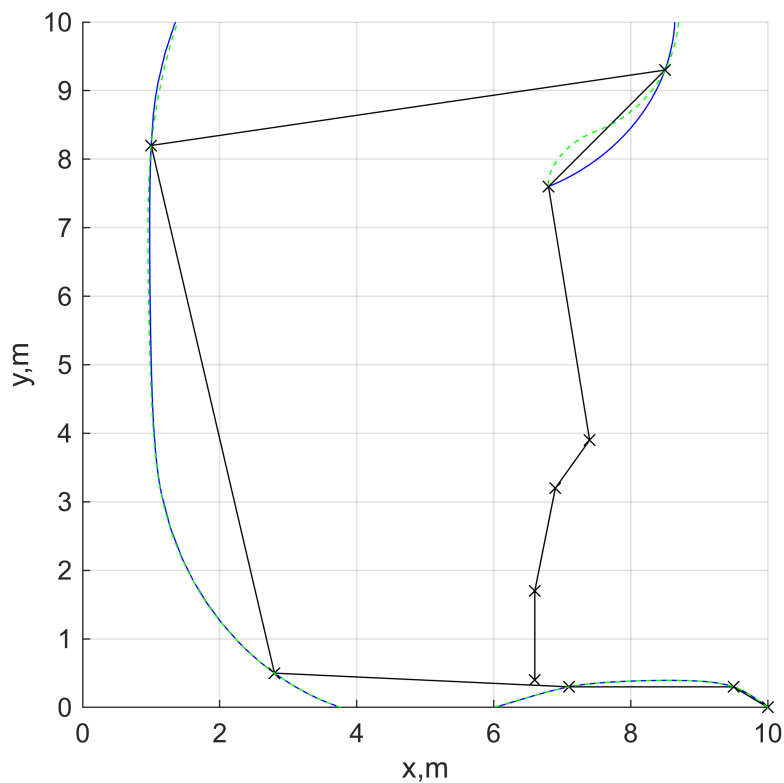
```
figure();
hold on
x = [sol.y(1,:),sol.y(9,:),sol.y(17,:),sol.y(25,:),sol.y(33,:),sol.y(41,:)]; % xsln
y = [sol.y(2,:),sol.y(10,:),sol.y(18,:),sol.y(26,:),sol.y(34,:),sol.y(42,:)]; % y
sln
plot(x,y,'b-') % plot sln
plot([origin(1,sequence(Goodx,1)),cities(1,sequence(Goodx,2:11))],...
[origin(2,sequence(Goodx,1)),cities(2,sequence(Goodx,2:11))],'-kx') % straight
line path
xlabel('x,m') % label x axis
ylabel('y,m') % label y axis
grid on % grid on for readability
axis equal % x and y axis equal
axis([0 10 0 10]) % set axis limits
```

Homotopy to reduce curvature

```
sol1 = sol;
for thetaf = -2.6:0.1:-pi/2+0.2
    sol1 =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc2(so,sf,times,cities,r,
,Goodx,sequence,thetaf,originx,originy),sol1);
end
sol = sol1;
x = [sol.y(1,:) sol.y(9,:) sol.y(17,:) sol.y(25,:) sol.y(33,:) sol.y(41,:)]; %
homotopy x array
y = [sol.y(2,:) sol.y(10,:) sol.y(18,:) sol.y(26,:) sol.y(34,:) sol.y(42,:)]; %
homotopy y array
```

Plot of homotopy solution

```
plot(x,y,'g--') % plot homotopy
```



Segment IV: City 6 to City 7 and 8

GA guess of switching times and lambdas

$\lambda_x(0)$ $\lambda_y(0)$ $\lambda_x(t_1^+)$ $\lambda_y(t_1^+)$ t_1 t_2

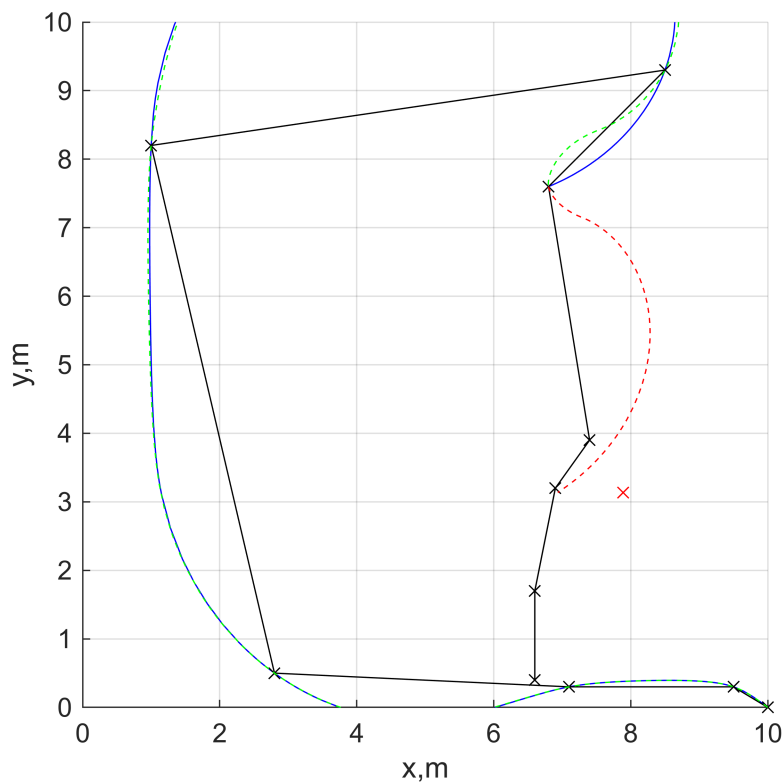
```
% options = optimoptions('ga','UseParallel',true,'MaxGenerations',1000);
% x4 = ga(@(x)FITNESS4(x,citySeq,r,[sol.y(end-5:end,end)]'),6,[],[],[],[],
[-10*ones(1,4),0.1,0.1],[10*ones(1,4),10,10],[],[],options) %beq and lb
% x4 = [-1.9307    -1.3585     2.4918    -4.1399     2.7802     0.4786]; % r = 2
% x4 = [-1.2222    -1.2096     0.5959    -0.1993     1.8128     1.3702]; % r = 100
x4 = [-1.2102    -1.1990     1.8516    -2.8625     2.7810     0.4438]; % r = 100
```

Generation of states guess

```
[T7,S7] = ode45(@(t,s)ELEOM(t,s,r),[0,x4(5)],[[sol.y(end-7:end-4,end)]',x4(1:2),
[sol.y(end-1:end,end)]']); %guess to city 7
[T8,S8] = ode45(@(t,s)ELEOM(t,s,r),[0,x4(6)],[S7(end,1:4),x4(3:4),S7(end,7:8)]); %
guess to city 8
```

Plot of guess

```
plot([S7(:,1);S8(:,1)],[S7(:,2);S8(:,2)], '--r') % plot guess for 6-7-8
plot(S7(end,1),S8(end,2), 'xr') % plot cities
hold off
```



BVP4C solution generation

```
solinit.x = linspace(0,1,21);
```

Interpolation of next two arcs

```
arc7 = interp1(T7,S7,solinit.x*T7(end)); % interpolate arc 7
arc8 = interp1(T8,S8,solinit.x*T8(end)); % interpolate arc 8
arc56soly = interp1(arc56solx', arc56soly', solinit.x*arc56solx(end)'); % create
states matrix for 4-5-6
```

Appending states guess to previous solution

```
solinit.y = [arc56soly,arc7,arc8]';
```

Appending times

```
solinit.parameters = [arc56solp(1:6)',T7(end),T8(end)];
```

BVP4C Solution for 6-7-8

```
sol =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc(so,sf,times,cities,r,
Goodx,sequence,originx,originy),solinit); % BVP4C of 6-7-8 sln
arc78solx = sol.x % times of 6-7-8 sln
```

```
arc78solx = 1x65
```

0 0.0083 0.0167 0.0250 0.0333 0.0417 0.0500 0.0583 ...

```
arc78soly = sol.y; % states of 6-7-8 sln
arc78solp = sol.parameters; % times of 6-7-8 sln
```

Plot of solution

```
figure();
hold on
x =
[sol.y(1,:),sol.y(9,:),sol.y(17,:),sol.y(25,:),sol.y(33,:),sol.y(41,:),sol.y(49,:),s
ol.y(57,:)]; % create x array of path
y =
[sol.y(2,:),sol.y(10,:),sol.y(18,:),sol.y(26,:),sol.y(34,:),sol.y(42,:),sol.y(50,:),
sol.y(58,:)]; % create y array of path
plot(x,y,'b-') % plot straight line path to cities
plot([origin(1,sequence(Goodx,1)),cities(1,sequence(Goodx,2:11))],...
      [origin(2,sequence(Goodx,1)),cities(2,sequence(Goodx,2:11))],'-kx') % plot sln
to city 8
xlabel('x,m') % label x axis
ylabel('y,m') % label y axis
grid on % grid on for readability
axis equal % set x and y axis equal
axis([0 10 0 10]) % set axis limits
TotalTime = sum(arc78solp) % total time to 8th city
```

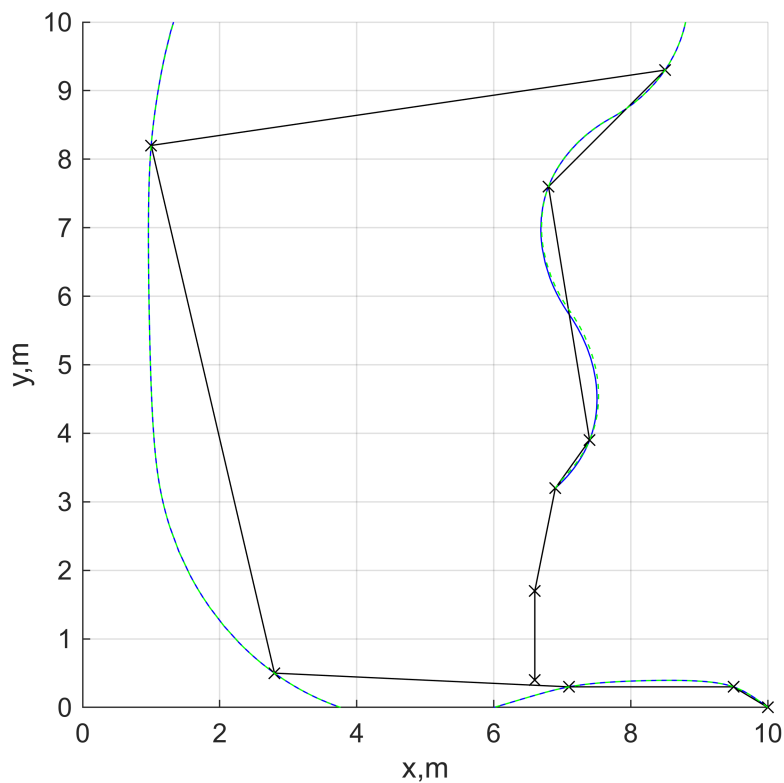
```
TotalTime =
13.0236
```

Homotopy to reduce curvature

```
sol1 = sol;
for thetaf = -2.4:0.1:-2.2
    sol1 =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc2(so,sf,times,cities,r
,Goodx,sequence,thetaf,originx,originy),sol1);
end
sol = sol1;
x = [sol.y(1,:) sol.y(9,:) sol.y(17,:) sol.y(25,:) sol.y(33,:) sol.y(41,:)
sol.y(49,:) sol.y(57,:)]; % create x array
y = [sol.y(2,:) sol.y(10,:) sol.y(18,:) sol.y(26,:) sol.y(34,:) sol.y(42,:)
sol.y(50,:) sol.y(58,:)]; % create y array
```

Plot of homotopy solution

```
plot(x,y,'g--')
```



Segment V: City 8 to City 9 and 10

GA guess of switching times and lambdas

$\lambda_x(0)$ $\lambda_y(0)$ $\lambda_x(t_1^+)$ $\lambda_y(t_1^+)$ t_1 t_2

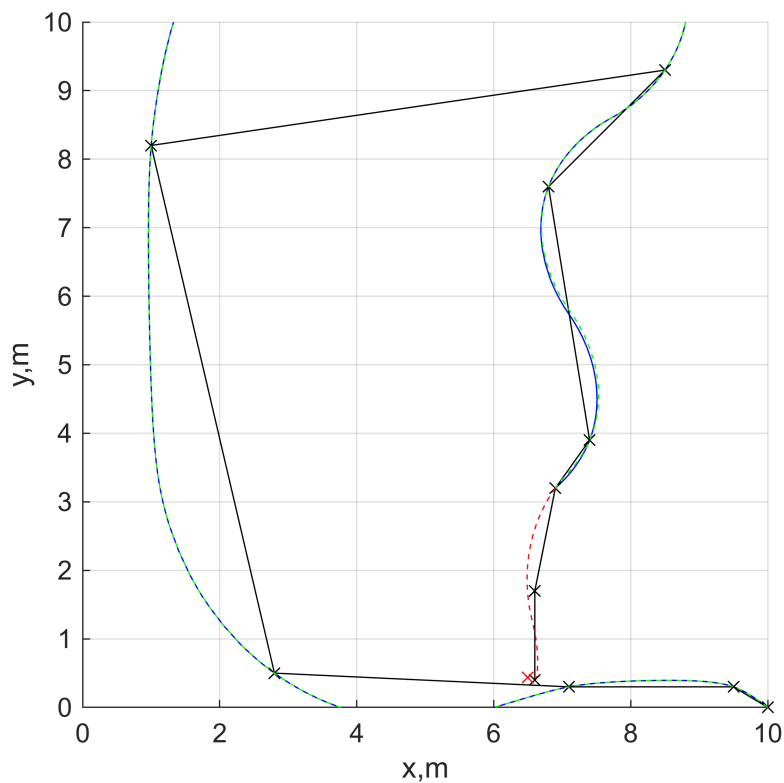
```
% options = optimoptions('ga','UseParallel',true,'MaxGenerations',1000);
% x5 = ga(@(x)FITNESS5(x,citySeq,r,[sol.y(end-5:end,end)]'),6,[],[],[],[],
[-10*ones(1,4),0.1,0.1],[10*ones(1,4),10,10],[],[],options) %beq and lb
% x5 = [0.1041    -0.0991    1.2423    -1.2420    5.0837    0.1000]; % r = 2
x5 = [-0.4080    0.1223    -0.2055    0.3405    0.7099    0.5523]; % r = 100
```

Generation of states guess

```
[T9,S9] = ode45(@(t,s)ELEOM(t,s,r),[0,x5(5)],[[sol.y(end-7:end-4,end)]',x5(1:2),
[sol.y(end-1:end,end)]']');%generating guess to 9th city
[T10,S10] = ode45(@(t,s)ELEOM(t,s,r),[0,x5(6)],[S9(end,1:4),x5(3:4),S9(end,7:8)]);
%generating guess to 10th city
```

Plot of guess to 9th and 10th cities

```
plot([S9(:,1);S10(:,1)],[S9(:,2);S10(:,2)], '--r')
plot(S9(end,1),S10(end,2), 'xr')
hold off
```



BVP4C solution generation

```
solinit.x = linspace(0,1,21);
```

Interpolation of next two arcs

```
arc9 = interp1(T9,S9,solinit.x*T9(end));
arc10 = interp1(T10,S10,solinit.x*T10(end));
arc78soly = interp1(arc78solx', arc78soly', solinit.x*arc78solx(end)');
```

Appending states guess to previous solution

```
solinit.y = [arc78soly,arc9,arc10]';
```

Appending times

```
solinit.parameters = [arc78solp(1:8)',T9(end),T10(end)];
```

BVP4c solution to 9th and 10th cities

```
sol =
bvp4c(@(t,s,times)ELEOM_Main(t,s,times,r),@(so,sf,times)bcfunc(so,sf,times,cities,r,
Goodx,sequence,originx,originy),solinit); %BVP4C of final solution
arc910solx = sol.x; % x for 8-9-10
arc910soly = sol.y; % states for 8-9-10
arc910solp = sol.parameters % times to 8-9-10
```



```

arc910solp = 10x1
    1.0823
    1.3669
    1.4256
    1.7425
    3.7902
    1.2283
    2.1128
    0.4555
    0.6377
    0.4372

```

Plot of total solution to all 10 cities.

```

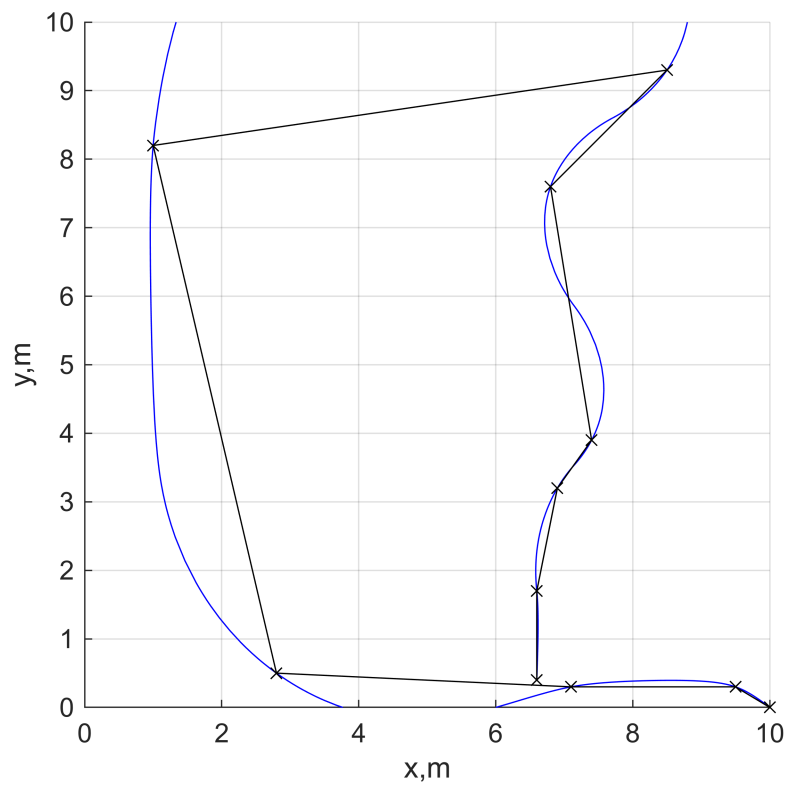
figure();
hold on

x = [];
y = [];
vel = [];

for i = 1:8:73
    x = [x, arc910soly(i, :)];
    y = [y, arc910soly(i+1, :)];
    vel = [vel, arc910soly(i+2, :)];
end

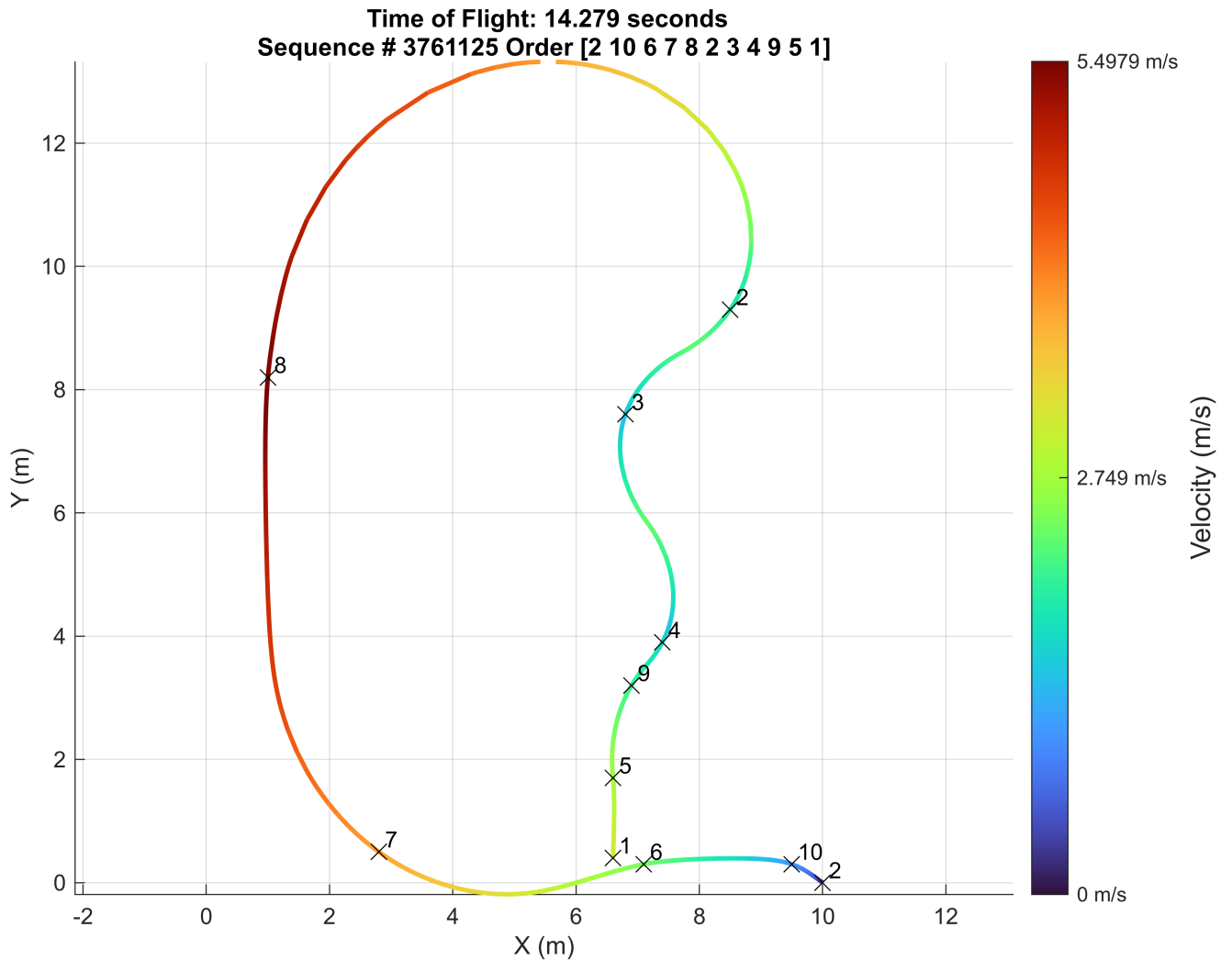
plot(x,y,'b-')
plot([origin(1,sequence(Goodx,1)),cities(1,sequence(Goodx,2:11))],...
     [origin(2,sequence(Goodx,1)),cities(2,sequence(Goodx,2:11))], '-kx')
xlabel('x,m')
ylabel('y,m')
grid on
axis equal
axis([0 10 0 10])
hold off

```



```
TotalTime = sum(arc910solp);
finalPlot(Goodx,x,y, citiesx, citiesy, sequence, vel, TotalTime)

xlim([-2.13 13.09])
ylim([-0.19 13.32])
```



The final solution to all 10 cities is plotted with a heat map jet overlay to represent velocity at each point.

```
TotalTime = sum(arc910solp) % total time to 10 cities, seconds
```

```
TotalTime =  
14.2790
```

Plot of control variables, alpha and gamma, vs time:

```
soly = [sol.y(1:8, :) sol.y(9:16, :) sol.y(17:24, :) sol.y(25:32, :)  
sol.y(33:40, :)...  
sol.y(41:48, :) sol.y(49:56, :) sol.y(57:64, :) sol.y(65:72, :)  
sol.y(73:80, :)] % combine solutions
```

```
Time = sol.x.*arc910solp;  
Time = [Time(1, :), Time(1,end) + Time(2,:), sum(Time(1:2,end)) + Time(3,:),  
sum(Time(1:3,end)) + Time(4,:),...  
sum(Time(1:4,end)) + Time(5,:), sum(Time(1:5,end)) + Time(6,:),  
sum(Time(1:6,end)) + Time(7,:),...]
```

```

sum(Time(1:7,end)) + Time(8,:), sum(Time(1:8,end)) + Time(9,:),
sum(Time(1:9,end)) + Time(10,:)]];

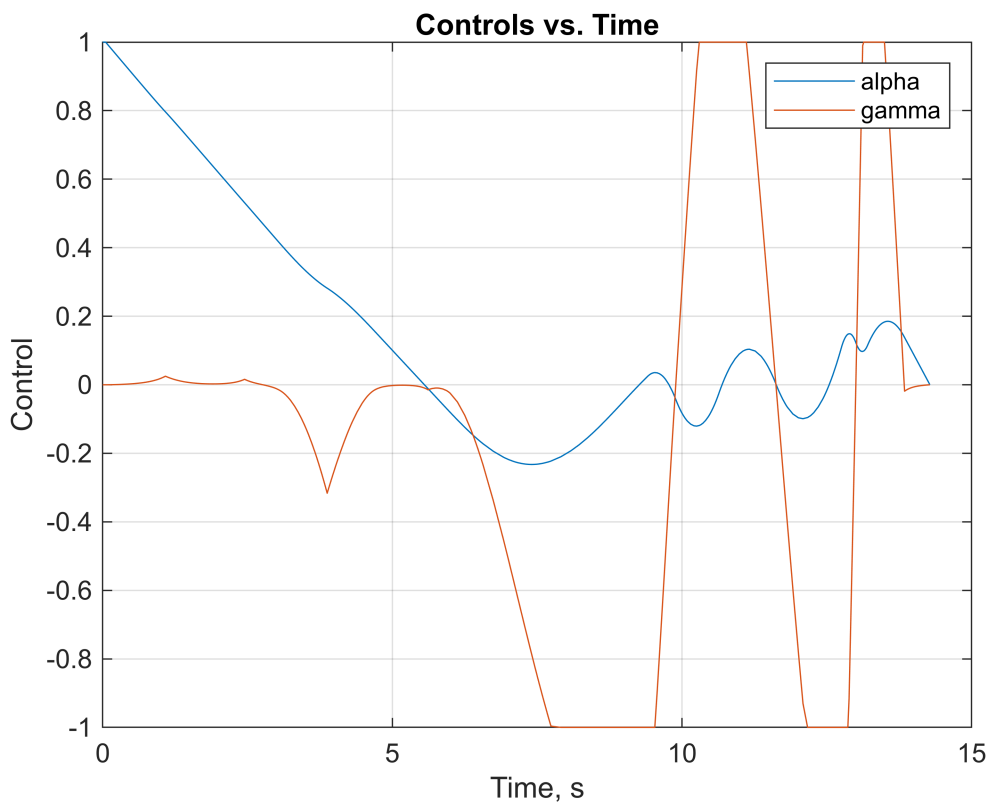
a = -r.*(soly(7,:))./r;% solve for alpha
gamma = -r.*(soly(8,:))./r;% solve for gamma

for i = 1:length(a)
    if abs(a(i))>1
        a(i) = sign(a(i));
    end
end

for i = 1:length(gamma)
    if abs(gamma(i))>1
        gamma(i) = sign(gamma(i));
    end
end

figure()
plot(Time,a,Time,gamma) % plot controls vs time
legend('alpha', 'gamma') % legend for readability
title('Controls vs. Time') % title of figure
xlabel('Time, s') % label x axis
ylabel('Control') % label y axis
grid on % grid on for readability

```



It is interesting to note how for almost the entire trajectory the acceleration is not fully used and instead hovers just barely positive.

Turning rate switching so aggressively back and forth makes sense as saturating turning is viable for reaching the next n targets as it won't cause

issues like overshoot.

Subfunctions

GAMULTIOBJ Subfunction to express fitness functions of distance and turning

```
function costs = FITNESSfunc(x,sequence,cities,origin)
d = (cities(1,sequence(x,2))-origin(1,sequence(x,1)))^2 +...
    (cities(2,sequence(x,2))-origin(2,sequence(x,1)))^2; %distance between given
cities

vec1 = [cities(1,sequence(x,2))-origin(1,sequence(x,1));...
        cities(2,sequence(x,2))-origin(2,sequence(x,1))]; % vector from one city to next

vec2 = [cities(1,sequence(x,3))-cities(1,sequence(x,2));...
        cities(2,sequence(x,3))-cities(2,sequence(x,2))]; % vector from one city to next

turn = acos((vec1(1)*vec2(1)+vec1(2)*vec2(2))/(norm(vec1)*norm(vec2))); % determine
angle between vectors

for k = 1:9
    d = (cities(1,sequence(x,k+2))-cities(1,sequence(x,k+1)))^2 +...
        (cities(2,sequence(x,k+2))-cities(2,sequence(x,k+1)))^2 + d; % distance
from each city for rest of sequence and add
end

for k = 1:8
    vec1 = vec2;
    vec2 = [cities(1,sequence(x,k+3))-cities(1,sequence(x,k+2));...
            cities(2,sequence(x,k+3))-cities(2,sequence(x,k+2))]; % determine vector to
next city
    turn = acos((vec1(1)*vec2(1)+vec1(2)*vec2(2))/(norm(vec1)*norm(vec2))) + turn;
% next angle of turn and add to last angle
end

costs = [d,turn]; % matrix of costs for each sequence

end
```

Euler-Lagrange EOMs Main Function

```
function dS = ELEOM_Main(t,s,times,r)
for k = 1:length(times)
```

```

    dS((1+(k-1)*8):8*k,1) = times(k)*ELEOM(t,s((1+(k-1)*8):8*k),r); %mult out for
efficiency
end
end

```

Euler-Lagrange EOMs for a single arc

```

function dS = ELEOM(~,s,r)
%s(1) = x s(2) = y s(3) = v s(4) = theta s(5) = lambda_x s(6) = lambda_y...
a = -r*s(7)/2;% solve for alpha

if abs(a)>1
    a = sign(a);% change the sign of alpha if it is greater than + or - 1
end

gamma = -r*s(8)/2;% solve for gamma
if abs(gamma)>1
    gamma = sign(gamma); % change the sign of gamma if it is greater than + or - 1
end

dS(1,1) = s(3)*cos(s(4)); % xdot = v*cos(theta)
dS(2,1) = s(3)*sin(s(4));% ydot = v*sin(theta)
dS(3,1) = a; % vdot = a
dS(4,1) = gamma; % thetadot = gamma
dS(5,1) = 0; % lambdaxdot = 0
dS(6,1) = 0;% lambdaydot = 0
dS(7,1) = -(s(5)*cos(s(4))+s(6)*sin(s(4)));% lambdavidot = -lambdax * cos(theta) -
lambday * sin(theta)
dS(8,1) = s(3)*(s(5)*sin(s(4))-s(6)*cos(s(4))); %lambdathetadot = v * (lambdax *
sin(theta) - lambday * cos(theta))

end

```

Boundary Condition Function

```

function con = bcfunc(so,sf,times,cities,r,x,sequence,originx,originy)

con = zeros(9*length(times),1);% create constraint array
con(1) = so(1)-originx; %origin x
con(2) = so(2)-originy; %origin y
con(3) = so(3); %v(0) = 0
con(4) = so(8); %theta = free --> lambda_theta(0) = 0

for k = 1:(length(times)-1)
    con((5+9*(k-1)):(8+9*(k-1))) = sf((1+8*(k-1)):(4+8*(k-1))) - so((1+8*k):
(4+8*k)); %states continuous
    con(9+9*(k-1)) = sf(1+8*(k-1)) - cities(1,sequence(x,k+1)); %city x
    con(10+9*(k-1)) = sf(2+8*(k-1)) - cities(2,sequence(x,k+1)); %city y
    con((11+9*(k-1)):(12+9*(k-1))) = sf((7+8*(k-1)):(8+8*(k-1))) - so((7+8*k):
(8+8*k)); %lambda_v and lambda theta continuous
end

```

```

    con(13+9*(k-1)) = Hfunc(sf((1+8*(k-1)):(8+8*(k-1))),r); %Transverslity condition
end

con(end-4) = sf(end-7) - cities(1,sequence(x,length(times)+1));% x end at the next
city
con(end-3) = sf(end-6) - cities(2,sequence(x,length(times)+1));%y end at the next
city
con(end-2) = sf(end-1); % lambda_v(tf) = 0
con(end-1) = sf(end); % lambda_theta(tf) = 0
con(end) = Hfunc(sf((end-7):end),r);% H(tf)= 0

end

```

Boundary Condition Function 2

```

function con = bcfunc2(so,sf,times,cities,r,x,sequence,thetaf,originx,originy)

con = zeros(9*length(times),1); % create constraint array
con(1) = so(1)-originx; %origin x
con(2) = so(2)-originy; %origin y
con(3) = so(3); %v(0) = 0
con(4) = so(8); %theta = free --> lambda_theta(0) = 0

for k = 1:(length(times)-1)
    con((5+9*(k-1)):(8+9*(k-1))) = sf((1+8*(k-1)):(4+8*(k-1))) - so((1+8*k):
(4+8*k)); %states continuous
    con(9+9*(k-1)) = sf(1+8*(k-1)) - cities(1,sequence(x,k+1)); %city x
    con(10+9*(k-1)) = sf(2+8*(k-1)) - cities(2,sequence(x,k+1)); %city y
    con((11+9*(k-1)):(12+9*(k-1))) = sf((7+8*(k-1)):(8+8*(k-1))) - so((7+8*k):
(8+8*k)); %lambda_v and lambda_theta continuous
    con(13+9*(k-1)) = Hfunc(sf((1+8*(k-1)):(8+8*(k-1))),r); %Transverslity condition
end

con(end-4) = sf(end-7) - cities(1,sequence(x,length(times)+1)); % x end at the next
city
con(end-3) = sf(end-6) - cities(2,sequence(x,length(times)+1)); %y end at the next
city
con(end-2) = sf(end-1); % lambda_v(tf) = 0
con(end-1) = sf(end-4)-thetaf; % lambda_theta(tf) = thetaf
con(end) = Hfunc(sf((end-7):end),r); % H(tf)= 0

end

```

Hamiltonian Subfunction

```

function H = Hfunc(s,r)

a = -r*s(7)/2; % solve for alpha
if abs(a)>1
    a = sign(a);% change the sign of alpha if it is greater than + or - 1
end

```

```

end

gamma = -r*s(8)/2; % solve for gamma
if abs(gamma)>1
    gamma = sign(gamma); % saturates control value at 1 or negative 1.
end

H = 1+(1/r)*(a^2+gamma^2)+s(5)*s(3)*cos(s(4))+s(6)*s(3)*sin(s(4))
+s(7)*a+s(8)*gamma; %solve for hamiltonian

end

```

Fitness Function 1 (Origin to First Two Cities)

```

function con = FITNESS1(x,cities,r)
[~,S1] = ode45(@(t,s)ELEOM(t,s,r),[0,x(7)],[cities(1,1),cities(2,1),0,x(1:4),0]);%
Determine states to the 1st city
[~,S2] = ode45(@(t,s)ELEOM(t,s,r),[0,x(8)],[S1(end,1:4),x(5:6),S1(end,7:8)]);%
Determine states to the 2nd city

distance1 = (S1(end,1)-cities(1,2))^2+(S1(end,2)-cities(2,2))^2;% determine how
close to the 1st city it got
distance2 = (S2(end,1)-cities(1,3))^2+(S2(end,2)-cities(2,3))^2;% determine how
close to the 2nd city it got

H1 = Hfunc(S1(end,:),r);% determining H at end of 1st city
H2 = Hfunc(S2(end,:),r);% determining H at end of 2nd city

con =
x(7)+x(8)+11*distance1+11*distance2+3*S2(end,7)^2+3*S2(end,8)^2+3*H1^2+3*H2^2;%
weighting of each constraint
end

```

Fitness Function 2 (City Visited 2 to City 3 then City 4)

```

function con = FITNESS2(x,cities,r,states)

[~,S1] = ode45(@(t,s)ELEOM(t,s,r),[0,x(5)],
[cities(1,3),cities(2,3),states(1:2),x(1:2),states(5:6)]);% Determine states to the
3rd city
[~,S2] = ode45(@(t,s)ELEOM(t,s,r),[0,x(6)],[S1(end,1:4),x(3:4),S1(end,7:8)]);%
Determine states to the 4th city

distance1 = (S1(end,1)-cities(1,4))^2+(S1(end,2)-cities(2,4))^2;% determine how
close to the 3rd city it got
distance2 = (S2(end,1)-cities(1,5))^2+(S2(end,2)-cities(2,5))^2;% determine how
close to the 4th city it got

H1 = Hfunc(S1(end,:),r);% determining H at end of 3rd city
H2 = Hfunc(S2(end,:),r);% determining H at end of 4th city

```



```

con = x(5)+x(6)+1*distance1+1*distance2+9*S2(end,7)^2+9*S2(end,8)^2+9*H1^2+9*H2^2;%
weighting of each constraint

end

```

Fitness Function 3 (City Visited 4 to City 5 then City 6)

```

function con = FITNESS3(x,cities,r,states)
[~,S1] = ode45(@(t,s)ELEOM(t,s,r),[0,x(5)],
[cities(1,5),cities(2,5),states(1:2),x(1:2),states(5:6)]);% Determine states to the
5th city
[~,S2] = ode45(@(t,s)ELEOM(t,s,r),[0,x(6)], [S1(end,1:4),x(3:4),S1(end,7:8)]);%
Determine states to the 6th city

distance1 = (S1(end,1)-cities(1,6))^2+(S1(end,2)-cities(2,6))^2;% determine how
close to the 5th city it got
distance2 = (S2(end,1)-cities(1,7))^2+(S2(end,2)-cities(2,7))^2;% determine how
close to the 6th city it got

H1 = Hfunc(S1(end,:),r); % determining H at end of 5th city
H2 = Hfunc(S2(end,:),r); % determining H at end of 6th city

con =
x(5)+x(6)+11*distance1+11*distance2+3*S2(end,7)^2+3*S2(end,8)^2+3*H1^2+3*H2^2; %
weighting of each constraint

end

```

Fitness Function 4 (City Visited 6 to City 7 then City 8)

```

function con = FITNESS4(x,cities,r,states)
[~,S1] = ode45(@(t,s)ELEOM(t,s,r),[0,x(5)],
[cities(1,7),cities(2,7),states(1:2),x(1:2),states(5:6)]); % Determine states to
the 7th city
[~,S2] = ode45(@(t,s)ELEOM(t,s,r),[0,x(6)], [S1(end,1:4),x(3:4),S1(end,7:8)]); %
Determine states to the 8th city

distance1 = (S1(end,1)-cities(1,8))^2+(S1(end,2)-cities(2,8))^2;% determine how
close to the 7th city it got
distance2 = (S2(end,1)-cities(1,9))^2+(S2(end,2)-cities(2,9))^2;% determine how
close to the 8th city it got

H1 = Hfunc(S1(end,:),r); % determining H at end of 7th city
H2 = Hfunc(S2(end,:),r); % determining H at end of 8th city

con =
x(5)+x(6)+11*distance1+11*distance2+3*S2(end,7)^2+3*S2(end,8)^2+3*H1^2+3*H2^2; %
weighting of each constraint

```

```
end
```

Fitness Function 5 (City Visited 8 to City 9 then City 10)

```
function con = FITNESS5(x,cities,r,states)
[~,S1] = ode45(@(t,s)ELEOM(t,s,r),[0,x(5)],
[cities(1,9),cities(2,9),states(1:2),x(1:2),states(5:6)]); % Determine states to
the 9th city
[~,S2] = ode45(@(t,s)ELEOM(t,s,r),[0,x(6)], [S1(end,1:4),x(3:4),S1(end,7:8)]); %
Determine states to the 10th city

distance1 = (S1(end,1)-cities(1,10))^2+(S1(end,2)-cities(2,10))^2; % determine how
close to the 9th city it got
distance2 = (S2(end,1)-cities(1,11))^2+(S2(end,2)-cities(2,11))^2;% determine how
close to the 10th city it got

H1 = Hfunc(S1(end,:),r);% determining H at end of 9th city
H2 = Hfunc(S2(end,:),r); % determining H at end of 10th city

con =
x(5)+x(6)+11*distance1+11*distance2+3*S2(end,7)^2+3*S2(end,8)^2+3*H1^2+3*H2^2; %
weighting of each constraint

end
```

Final Plotting Function

This function plots the heat map of velocities on the trajectory

```
function finalPlot(seqNum,x1,y1, citiesx, citiesy, sequence, velocities, time)
order = sequence(seqNum,:);
norm_velocities = velocities./max(velocities);
colormap_turbo = turbo(256);
colors = interp1(linspace(0, 1, 256), colormap_turbo, norm_velocities); % Map
velocities to colors
figure();
hold on

for i = 1:(length(x1))-1
    %start and end points for the segment
    x = x1(i:i+1);
    y = y1(i:i+1);

    % Plot each segment with a color corresponding to velocity, this is the
    % biggest issue I had in getting this to size right.
    plot(x,y, 'LineWidth', 2, 'Color', colors(i, :))
end

for i = 1:length(order)
    plot(citiesx(i), citiesy(i) , 'Xk', 'MarkerSize', 10); % Mark point with 'X'
```

```

    text(citiesx(i) + 0.1, citiesy(i)+.2, sprintf('%d', order(i)), 'FontSize', 10,
'Color', 'k'); % Annotate order
end

grid on
axis([0 12 0 12])
axis equal

cb = colorbar('Location','eastoutside');
colormap(turbo);
clim([min(velocities), max(velocities)]); % Label colorbar with velocity range
cb.Ticks = [0, 0.5*max(velocities), max(velocities)];
cb.TickLabels = {
    '0 m/s', ...
    [num2str(max(velocities)*0.5), ' m/s'], ...
    [num2str(max(velocities)), ' m/s']}
};
ylabel(cb, 'Velocity (m/s)', 'FontSize', 12);
xlabel('X (m)')
ylabel('Y (m)')
title([' Time of Flight: ', num2str(time), ' seconds'], ['Sequence #
',int2str(seqNum),' Order ', mat2str(order)])
set(gcf, 'Position', [100, 100, 800, 600]);
end

```